

C++ Programming

Day 5: 구조체와 프로젝트 분리

구조체, 열거형, 이름 공간, 헤더 파일, 다중 소스 파일

Contents

1	학습 목표	3
2	구조체: struct	3
2.1	구조체 정의	3
2.2	구조체 변수 생성과 멤버 접근	3
2.3	중괄호 초기화	4
2.4	구조체를 함수에 전달하기	4
2.5	전체 예제	4
3	열거형: enum	5
3.1	기본 열거형	5
3.2	값 직접 지정하기	5
3.3	enum class	5
3.4	조건문과 함께 사용하기	6
4	이름 공간: namespace	6
4.1	이름 충돌의 예	6
4.2	이름 공간 정의	6
4.3	같은 이름의 함수 구분하기	7
4.4	using namespace	7
5	선언과 정의	7
5.1	선언	7
5.2	정의	8
5.3	선언이 필요한 이유	8
6	헤더 파일	8
6.1	기본 파일 구성	8
6.2	헤더 파일	8
6.3	구현 파일	9
6.4	메인 파일	9
6.5	따옴표와 꺾쇠괄호	9
7	Include Guard	9
7.1	기본 형식	9
7.2	#pragma once	10
8	여러 소스 파일로 프로젝트 구성하기	10
8.1	프로젝트 구조	10
8.2	수학 모듈	10
8.3	학생 모듈	11
8.4	메인 파일	11
8.5	컴파일	12
8.6	컴파일과 링크	12
8.7	링커 오류	12
9	실습 프로젝트	12
9.1	폴더 구조	12
9.2	계산 모듈	13
9.3	정보 출력 모듈	13
9.4	메인 파일	13

9.5 실행 결과	14
9.6 컴파일 명령	14
10 폴더 이동과 와일드카드	14
11 실습 목록	15
12 핵심 정리	15

1. 학습 목표

학습 목표

이 장을 마치면 다음 내용을 설명하고 직접 코드로 작성할 수 있다.

- 여러 자료형을 하나의 구조체로 묶기
- 열거형을 사용하여 의미 있는 상태 표현하기
- 이름 공간으로 이름 충돌 방지하기
- 함수 선언과 정의를 분리하기
- 헤더 파일과 소스 파일의 역할 구분하기
- include guard로 중복 포함 방지하기
- 여러 .cpp 파일로 구성된 프로그램 컴파일하기

2. 구조체: struct

구조체는 서로 관련된 여러 데이터를 하나의 새로운 자료형으로 묶는 기능이다. 학생 한 명의 정보를 저장한다고 생각해 보자.

```
1 string name;
2 int age;
3 double gpa;
```

각 변수를 따로 선언할 수도 있지만, 학생 수가 늘어나면 어떤 값들이 한 학생에게 속하는지 관리하기 어려워진다. 구조체를 사용하면 관련 데이터를 하나의 단위로 표현할 수 있다.

2.1. 구조체 정의

```
1 #include <iostream>
2 #include <string>
3
4 using namespace std;
5
6 struct Student
7 {
8     string name;
9     int age;
10    double gpa;
11 };
```

Student는 사용자가 직접 정의한 새로운 자료형이다. 구조체 내부의 변수 name, age, gpa를 멤버(member)라고 한다.

핵심 포인트

구조체 정의 뒤에는 반드시 세미콜론을 작성해야 한다.

2.2. 구조체 변수 생성과 멤버 접근

```

1 Student student;
2
3 student.name = "Alice";
4 student.age = 20;
5 student.gpa = 4.1;

```

점 연산자 .를 사용하면 구조체의 멤버에 접근할 수 있다.

```

1 cout << student.name << endl;
2 cout << student.age << endl;
3 cout << student.gpa << endl;

```

2.3. 중괄호 초기화

```

1 Student student = {"Alice", 20, 4.1};

```

값은 구조체에 선언된 멤버 순서대로 저장된다.

```

1 struct Student
2 {
3     string name;
4     int age;
5     double gpa;
6 };

```

따라서 첫 번째 값은 name, 두 번째 값은 age, 세 번째 값은 gpa에 저장된다.

2.4. 구조체를 함수에 전달하기

```

1 void printStudent(const Student& student)
2 {
3     cout << "Name: " << student.name << endl;
4     cout << "Age : " << student.age << endl;
5     cout << "GPA : " << student.gpa << endl;
6 }

```

const Student&는 구조체 전체를 복사하지 않고 원본을 읽기 전용으로 전달한다.

2.5. 전체 예제

```

1 #include <iostream>
2 #include <string>
3
4 using namespace std;
5
6 struct Student
7 {
8     string name;
9     int age;
10    double gpa;
11 };
12
13 void printStudent(const Student& student)
14 {
15     cout << "Name: " << student.name << endl;
16     cout << "Age : " << student.age << endl;
17     cout << "GPA : " << student.gpa << endl;
18 }

```

```

19
20 int main()
21 {
22     Student student = {"Alice", 20, 4.1};
23
24     printStudent(student);
25
26     return 0;
27 }

```

3. 열거형: enum

열거형은 서로 관련된 이름 있는 정수 상수들을 하나의 자료형으로 묶는다. 프로그램의 상태를 단순한 숫자보다 읽기 쉬운 이름으로 표현할 때 사용한다.

3.1. 기본 열거형

```

1 enum Day
2 {
3     Monday,
4     Tuesday,
5     Wednesday,
6     Thursday,
7     Friday,
8     Saturday,
9     Sunday
10 };

```

기본적으로 첫 번째 원소는 0부터 시작하고 이후 값은 1씩 증가한다.

```

1 Day today = Wednesday;
2
3 cout << today << endl;

```

출력되는 정수 값은 2이다.

3.2. 값 직접 지정하기

```

1 enum Status
2 {
3     Pending = 1,
4     Running = 2,
5     Completed = 3
6 };

```

열거형 원소에 특정 정수 값을 직접 지정할 수 있다.

3.3. enum class

기본 enum의 원소 이름은 주변 영역에 그대로 노출된다. 큰 프로그램에서는 다른 열거형과 이름이 충돌할 수 있다.

```

1 enum class Color
2 {
3     Red,
4     Green,
5     Blue
6 };

```

```
7
8 Color color = Color::Red;
```

enum class에서는 Color::Red처럼 열거형 이름을 함께 작성해야 한다. 그 대신 이름 충돌을 줄이고 자료형 검사를 더 엄격하게 할 수 있다.

핵심 포인트

새로운 코드에서는 단순한 enum보다 enum class를 우선 고려하는 것이 좋다.

3.4. 조건문과 함께 사용하기

```
1 enum class Status
2 {
3     Pending,
4     Running,
5     Completed
6 };
7
8 int main()
9 {
10     Status status = Status::Running;
11
12     if (status == Status::Running)
13     {
14         cout << "Program is running." << endl;
15     }
16
17     return 0;
18 }
```

4. 이름 공간: namespace

서로 다른 코드에서 같은 이름을 사용하면 충돌이 발생할 수 있다. 이름 공간은 관련된 이름을 하나의 범위 안에 묶어 충돌을 방지한다.

4.1. 이름 충돌의 예

```
1 int add(int a, int b)
2 {
3     return a + b;
4 }
```

다른 라이브러리에도 같은 이름과 매개변수를 가진 add 함수가 있다면 컴파일러가 어느 함수를 사용해야 하는지 구분하기 어렵다.

4.2. 이름 공간 정의

```
1 namespace Math
2 {
3     int add(int a, int b)
4     {
5         return a + b;
6     }
7 }
```

이름 공간 안의 함수는 범위 지정 연산자 `::`로 접근한다.

```
1 cout << Math::add(10, 20) << endl;
```

4.3. 같은 이름의 함수 구분하기

```
1 namespace IntegerMath
2 {
3     int add(int a, int b)
4     {
5         return a + b;
6     }
7 }
8
9 namespace DoubleMath
10 {
11     double add(double a, double b)
12     {
13         return a + b;
14     }
15 }
16
17 int main()
18 {
19     cout << IntegerMath::add(10, 20) << endl;
20     cout << DoubleMath::add(1.5, 2.5) << endl;
21
22     return 0;
23 }
```

4.4. using namespace

```
1 using namespace std;
```

위 문장은 `std::cout` 대신 `cout`을 사용할 수 있게 한다. 작은 예제에서는 편리하지만 헤더 파일이나 큰 프로젝트에서는 이름 충돌 가능성을 높일 수 있다.

핵심 포인트

헤더 파일에서는 일반적으로 `using namespace std;`를 작성하지 않는다. 필요한 이름에 `std::`를 직접 붙이는 편이 안전하다.

5. 선언과 정의

여러 파일로 프로그램을 나누려면 선언과 정의의 차이를 이해해야 한다.

5.1. 선언

선언은 이름과 사용 방법을 컴파일러에 알려 준다.

```
1 int add(int a, int b);
```

이 문장은 `add`라는 함수가 존재하며 두 개의 `int`를 받아 `int`를 반환한다고 알린다. 함수 본문은 없기 때문에 실제 동작은 아직 작성되지 않았다.

5.2. 정의

정의는 함수의 실제 동작을 작성한다.

```
1 int add(int a, int b)
2 {
3     return a + b;
4 }
```

5.3. 선언이 필요한 이유

C++ 컴파일러는 코드를 위에서 아래로 읽는다.

```
1 int main()
2 {
3     cout << add(10, 20) << endl;
4
5     return 0;
6 }
7
8 int add(int a, int b)
9 {
10    return a + b;
11 }
```

main()을 읽는 시점에는 add가 아직 알려지지 않았으므로 오류가 발생한다. 앞부분에 함수 선언을 추가하면 해결된다.

```
1 int add(int a, int b);
2
3 int main()
4 {
5     cout << add(10, 20) << endl;
6
7     return 0;
8 }
9
10 int add(int a, int b)
11 {
12     return a + b;
13 }
```

6. 헤더 파일

헤더 파일은 여러 소스 파일에서 공유할 선언을 저장한다. 일반적으로 확장자는 .h 또는 .hpp를 사용한다.

6.1. 기본 파일 구성

```
04_header/
    04_main.cpp
    04_math.cpp
    04_math.h
```

6.2. 헤더 파일

```
04_math.h
```

```

1 #ifndef MATH_H
2 #define MATH_H
3
4 int add(int a, int b);
5
6 #endif

```

6.3. 구현 파일

04_math.cpp

```

1 #include "04_math.h"
2
3 int add(int a, int b)
4 {
5     return a + b;
6 }

```

구현 파일은 자신의 헤더를 포함하는 것이 좋다. 그러면 선언과 정의의 형태가 서로 다른 경우 컴파일러가 이를 발견할 수 있다.

6.4. 메인 파일

04_main.cpp

```

1 #include <iostream>
2 #include "04_math.h"
3
4 using namespace std;
5
6 int main()
7 {
8     cout << add(10, 20) << endl;
9
10    return 0;
11 }

```

6.5. 따옴표와 꺾쇠괄호

```

1 #include <iostream>
2 #include "04_math.h"

```

<iostream>처럼 꺾쇠괄호를 사용하면 표준 라이브러리나 시스템 경로에서 헤더를 찾는다. "04_math.h"처럼 따옴표를 사용하면 현재 프로젝트의 파일을 우선해서 찾는다.

7. Include Guard

헤더 파일이 여러 경로를 통해 반복해서 포함되면 같은 선언이나 구조체가 중복 처리될 수 있다. include guard는 헤더의 내용이 한 번만 포함되도록 한다.

7.1. 기본 형식

```

1 #ifndef STUDENT_H
2 #define STUDENT_H
3
4 struct Student

```

```

5 {
6     int id;
7 };
8
9 #endif

```

처리 과정은 다음과 같다.

1. STUDENT.H가 정의되어 있는지 확인한다.
2. 정의되어 있지 않으면 헤더 내용을 처리한다.
3. STUDENT.H를 정의한다.
4. 이후 같은 헤더를 다시 만나면 내용을 건너뛰다.

7.2. #pragma once

```

1 #pragma once
2
3 int add(int a, int b);

```

#pragma once도 파일을 한 번만 포함하도록 한다. 작성은 간단하지만 전통적인 표준 방식은 include guard이다.

핵심 포인트

include guard의 이름은 프로젝트 전체에서 겹치지 않도록 파일 이름을 기반으로 작성하는 것이 좋다. 예: P05_CALCULATOR.H

8. 여러 소스 파일로 프로젝트 구성하기

프로그램이 커지면 모든 코드를 하나의 .cpp 파일에 작성하기 어렵다. 기능별로 파일을 분리하면 코드의 역할이 명확해지고 수정과 재사용이 쉬워진다.

8.1. 프로젝트 구조

```

05_multiple_files/
    05_main.cpp
    05_math.cpp
    05_math.h
    05_student.cpp
    05_student.h

```

8.2. 수학 모듈

05_math.h

```

1 #ifndef MATH_H
2 #define MATH_H
3
4 int add(int a, int b);
5
6 #endif

```

05_math.cpp

```
1 #include "05_math.h"
2
3 int add(int a, int b)
4 {
5     return a + b;
6 }
```

8.3. 학생 모듈

05_student.h

```
1 #ifndef STUDENT_H
2 #define STUDENT_H
3
4 #include <string>
5
6 struct Student
7 {
8     std::string name;
9     int age;
10    double gpa;
11 };
12
13 void printStudent(const Student& student);
14
15 #endif
```

05_student.cpp

```
1 #include <iostream>
2 #include "05_student.h"
3
4 void printStudent(const Student& student)
5 {
6     std::cout << "Name: " << student.name << std::endl;
7     std::cout << "Age : " << student.age << std::endl;
8     std::cout << "GPA : " << student.gpa << std::endl;
9 }
```

8.4. 메인 파일

05_main.cpp

```
1 #include <iostream>
2 #include "05_math.h"
3 #include "05_student.h"
4
5 using namespace std;
6
7 int main()
8 {
9     Student student = {"Alice", 20, 4.1};
10
11     cout << add(10, 20) << endl;
12     printStudent(student);
13
14     return 0;
15 }
```

8.5. 컴파일

프로젝트 폴더 안에서 다음 명령을 실행한다.

```
g++ *.cpp -o 05_multiple_files
```

Windows에서는 다음 명령으로 실행할 수 있다.

```
.\05_multiple_files.exe
```

*.cpp는 현재 폴더에 있는 모든 .cpp 파일을 의미한다.

8.6. 컴파일과 링크

각 .cpp 파일은 별도로 컴파일된 뒤 하나의 실행 파일로 연결된다.

1. 전처리가 #include 내용을 처리한다.
2. 컴파일러가 각 .cpp 파일을 목적 파일로 변환한다.
3. 링커가 목적 파일들을 하나의 실행 파일로 결합한다.

8.7. 링커 오류

헤더에 선언만 있고 어떤 .cpp 파일에도 정의가 없으면 컴파일 단계는 통과할 수 있지만 링크 단계에서 오류가 발생한다.

```
1 int add(int a, int b);
```

위 선언을 사용했지만 정의를 작성하지 않았다면 다음과 유사한 오류가 발생한다.

```
undefined reference to 'add(int, int)'
```

또한 필요한 .cpp 파일을 컴파일 명령에서 빠뜨려도 같은 문제가 발생할 수 있다.

핵심 포인트

헤더 파일은 일반적으로 직접 컴파일하지 않는다. 헤더를 포함하는 여러 .cpp 파일을 함께 컴파일한다.

9. 실습 프로젝트

이번 실습은 계산 기능과 정보 출력 기능을 각각 다른 모듈로 분리한다.

9.1. 폴더 구조

```
practice/
  p05_multiple_files/
    p05_main.cpp
    p05_calculator.cpp
    p05_calculator.h
    p05_info.cpp
    p05_info.h
```

9.2. 계산 모듈

p05_calculator.h

```
1 #ifndef P05_CALCULATOR_H
2 #define P05_CALCULATOR_H
3
4 int add(int a, int b);
5 int multiply(int a, int b);
6
7 #endif
```

p05_calculator.cpp

```
1 #include "p05_calculator.h"
2
3 int add(int a, int b)
4 {
5     return a + b;
6 }
7
8 int multiply(int a, int b)
9 {
10    return a * b;
11 }
```

9.3. 정보 출력 모듈

p05_info.h

```
1 #ifndef P05_INFO_H
2 #define P05_INFO_H
3
4 void printInfo();
5
6 #endif
```

p05_info.cpp

```
1 #include <iostream>
2 #include "p05_info.h"
3
4 using namespace std;
5
6 void printInfo()
7 {
8     cout << "C++ Day 05 Practice" << endl;
9     cout << "Multiple File Project" << endl;
10 }
```

9.4. 메인 파일

p05_main.cpp

```
1 #include <iostream>
2 #include "p05_calculator.h"
3 #include "p05_info.h"
4
5 using namespace std;
6
7 int main()
```

```

8 {
9     int a = 10;
10    int b = 20;
11    int c = 4;
12    int d = 8;
13
14    cout << a << " + " << b << " = " << add(a, b) << endl;
15    cout << c << " * " << d << " = " << multiply(c, d) << endl;
16    cout << endl;
17
18    printInfo();
19
20    return 0;
21 }

```

9.5. 실행 결과

10 + 20 = 30

4 * 8 = 32

C++ Day 05 Practice
Multiple File Project

9.6. 컴파일 명령

```
g++ *.cpp -o p05_multiple_files
```

10. 폴더 이동과 와일드카드

터미널에서 현재 폴더의 부모 폴더로 이동하려면 다음 명령을 사용한다.

```
cd ..
```

두 단계 위의 폴더로 이동하려면 다음과 같이 작성한다.

```
cd ../../
```

여기서 점 두 개 ..는 부모 폴더를 의미한다. 슬래시로 이어 쓰면 여러 단계 위로 이동할 수 있다. 현재 폴더의 모든 C++ 소스 파일을 함께 컴파일할 때는 와일드카드 *를 사용한다.

```
g++ *.cpp -o program
```

*.cpp는 이름과 관계없이 확장자가 .cpp인 모든 파일을 의미한다.

11. 실습 목록

실습

Day 5 실습 파일은 다음과 같이 관리한다.

- `practice/p01_struct.cpp`
- `practice/p02_enum.cpp`
- `practice/p03_namespace.cpp`
- `practice/p04_header/p04_main.cpp`
- `practice/p04_header/p04_math.cpp`
- `practice/p04_header/p04_math.h`
- `practice/p05_multiple_files/p05_main.cpp`
- `practice/p05_multiple_files/p05_calculator.cpp`
- `practice/p05_multiple_files/p05_calculator.h`
- `practice/p05_multiple_files/p05_info.cpp`
- `practice/p05_multiple_files/p05_info.h`

12. 핵심 정리

- `struct`는 서로 관련된 여러 데이터를 하나의 자료형으로 묶는다.
- `enum class`는 프로그램의 상태를 의미 있는 이름으로 표현한다.
- `namespace`는 같은 이름을 안전하게 구분한다.
- 선언은 이름과 사용 방법을 알리고, 정의는 실제 동작을 제공한다.
- 헤더 파일에는 공유할 선언을 작성한다.
- 소스 파일에는 함수의 실제 정의를 작성한다.
- `include guard`는 헤더의 중복 포함을 방지한다.
- 여러 `.cpp` 파일은 함께 컴파일한 뒤 링커가 하나의 실행 파일로 결합한다.
- 기능별 파일 분리는 이후 클래스와 객체지향 프로그래밍을 배우기 위한 중요한 기반이다.

핵심 포인트

Day 5의 핵심은 단순히 파일 개수를 늘리는 것이 아니다. 하나의 프로그램을 역할에 따라 나누고, 각 파일이 무엇을 공개하고 무엇을 구현하는지 구분하는 습관을 익히는 것이다.